YourAlgo v2 — Master Antigravity Setup Prompt

Purpose: Complete dynamic Indian equity algo-trading platform specification. This document is intended to be pasted into Antigravity as the master implementation brief.

1. Product Identity & Objective

• Build **YourAlgo v2**, a production-oriented Indian equity algorithmic trading platform.

• Starting capital: ■10,000. Planned risk per trade: 1% (■100). Daily loss limit: ■200. Maximum open positions: 2. Maximum trades/day: 3. Default risk/reward: 1:3.

• Build paper trading first. Live trading must remain disabled/locked until validation, broker configuration, compliance checks and explicit user activation are complete.

• Do not create a static showcase. Every dashboard, table, chart, strategy, order, portfolio and report must be driven by backend/database state.

2. Technology Stack

• Frontend: React + Vite + TypeScript + Tailwind CSS.

• Backend: FastAPI + Python.

• Database: MySQL.

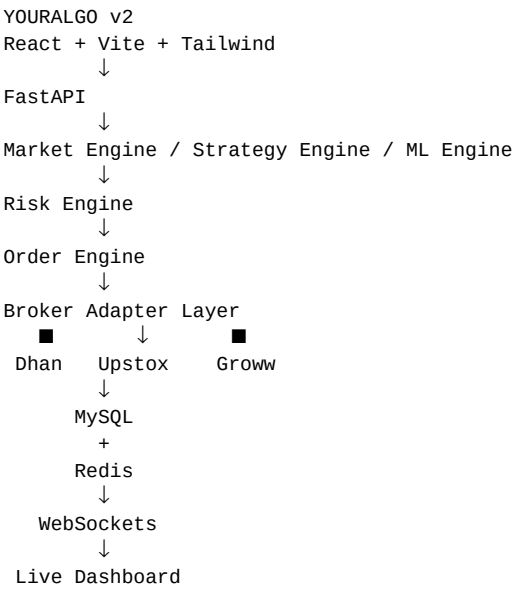
• Cache/realtime/queues/locks: Redis.

• ML/data: Python, Pandas, NumPy, scikit-learn.

• Realtime: WebSockets.

• Deployment: Vercel-compatible frontend, Docker-compatible backend/workers, managed MySQL and Redis.

3. High-Level Architecture



• Keep responsibilities separated. ML predicts probabilities; strategy creates candidates; risk decides whether a trade is allowed; order engine executes; broker adapters communicate with brokers; MySQL is permanent truth; Redis is temporary state/queue/lock/realtime infrastructure.

4. Public Website

• Landing page, Product, Strategies, Backtesting, How It Works, Pricing, Documentation, Login and Register.

• Use a professional fintech/terminal visual style with responsive desktop/tablet/mobile layouts.

• Clearly communicate that this is a trading tool and not a guaranteed-profit system.

5. Authentication & Security

• Implement registration, login, logout, email verification, password reset, session management and protected routes.

• Use short-lived access tokens and rotating refresh tokens. Use secure HTTP-only cookies where appropriate.

• Support Google OAuth through FastAPI. Never expose OAuth client secrets in the frontend.

• Support optional TOTP 2FA and backup codes.

• Use Argon2/bcrypt for passwords, Redis rate limiting, brute-force protection, account lockout, CSRF/CORS protections as appropriate, secure headers and input validation.

• RBAC roles: USER, PREMIUM_USER, ADMIN, SYSTEM.

• Permissions include VIEW_MARKET_DATA, RUN_BACKTEST, TRAIN_MODEL, PAPER_TRADE, CONNECT_BROKER, LIVE_TRADE, MANAGE_USERS and MANAGE_SYSTEM.

• Encrypt broker credentials/secrets at rest. Never log API secrets or tokens.

6. Database

```
Core entities should include:
users
sessions
refresh_tokens
oauth_accounts
two_factor_credentials
backup_codes
broker_connections
instruments
instrument_mappings
watchlists
watchlist_items
strategies
strategy_versions
strategy_rules
models
model_versions
datasets
training_runs
backtest_runs
backtest_trades
paper_orders
orders
order_events
trades
positions
portfolio_snapshots
risk_events
notifications
audit_logs
system_health
market_data_metadata
```

• Use migrations, foreign keys, indexes, timestamps and appropriate constraints. Executed strategy/model versions must be immutable and auditable.

7. Instrument Master

• Create an instrument master containing exchange, segment, symbol, security_id, instrument_token, ISIN, tradingsymbol, expiry, strike, option type, lot size and tick size where applicable.

• Maintain broker-specific mappings for Dhan, Upstox and Groww.

• Never place an order using an ambiguous symbol. Resolve instruments through the canonical instrument master.

8. Market Data Engine

• Build a provider abstraction for realtime and historical market data.

• Support LTP, OHLC, volume, timestamp, bid/ask/depth where available, daily high/low and historical candles according to provider capabilities.

• Validate missing data, duplicates, out-of-order timestamps, impossible OHLC values, volume anomalies, holidays and corporate-action effects.

• Mark stale feeds. If market data is stale, block new trading orders.

9. Realtime Infrastructure

• Use WebSockets for dashboard updates.

• Redis may handle pub/sub, short-lived state, queues and distributed locks.

• Browser disconnects must never trigger trading actions.

• Provide reconnect logic and clear connection status in the UI.

10. Dashboard

• Show live capital, available cash, P&L;, open positions, win rate, drawdown, risk used, market status, data-feed status, algo status and broker status.

• Use realtime updates instead of fake/static values.

• Display proper loading, empty, unavailable and error states.

11. Stock Terminal

• Show LTP/OHLC/volume and interactive candlestick + volume charts.

• Support EMA20/EMA50, RSI, MACD, ATR, VWAP and Bollinger Bands.

• Show support/resistance, strategy signals, entry/stop/target markers and available fundamentals/news/trade information.

• All values must come from configured data sources or clearly show unavailable status.

12. Watchlist & Screener

• Create editable watchlists.

• Build a dynamic screener supporting Market Cap, P/E, ROE, ROCE, Debt/Equity, revenue growth, profit growth, RSI, EMA, VWAP, ATR, relative volume, breakout, 52-week high and price-change conditions.

• Allow users to save, edit, delete, run and export screeners.

• Run screening on the backend and return paginated results.

13. Strategy Engine & Builder

• Create a rule-based strategy builder with indicators, comparisons, AND/OR groups, entry rules, exit rules, stop-loss and target rules.

```
Example:
EMA20 > EMA50
AND RSI > 50
AND Close > 20D High
AND RVOL > 1.5
THEN BUY
```

• Support ATR-based stop loss and target calculations. Save strategy versions. Never mutate the exact strategy definition used by an executed trade.

14. ML Lab

• Provide dataset selection, data cleaning, feature generation, labeling, training, validation, walk-forward testing and out-of-sample evaluation.

• Possible features: returns, volatility, RSI, MACD, EMA distances, ATR, VWAP distance, relative volume, breakout distance, market regime and time-of-day.

• Store model versions, training parameters, metrics, datasets and timestamps in the model registry.

• ML outputs probabilities/confidence; ML must never directly place an order.

15. Backtesting

• Support stock/universe, date range, strategy version, starting capital, risk per trade, RR, brokerage, taxes, fees and slippage.

• Calculate net P&L;, return, max drawdown, win rate, profit factor, Sharpe, Sortino, expectancy, trade count, average win/loss, holding time and turnover.

• Display equity curve, drawdown chart, monthly returns and trade distribution.

• Run large backtests asynchronously through workers rather than blocking HTTP requests.

• Include configurable Indian-market transaction costs such as brokerage, STT, exchange charges, GST, SEBI charges and stamp duty where applicable.

• Avoid survivorship bias and account for corporate actions/data quality.

16. Paper Trading

• Use real market data and real strategy/risk checks, but simulate fills.

• Simulate realistic slippage, spread, partial fills, rejection, cancellation and fees.

• Use the same strategy/risk/order interfaces as live trading; only the execution adapter changes.

17. Risk Engine

• Enforce 1% risk per trade, ■200 daily loss limit, maximum 2 open positions, maximum 3 trades/day and default 1:3 RR.

• Position sizing = floor(risk_amount / stop_distance), then validate capital, maximum position value, lot/tick size and broker constraints.

• Block trades when daily loss limit is reached, maximum positions/trades are reached, capital is insufficient, stop/target is invalid, risk is too high, position is too large, data is stale, market is closed, broker is disconnected, instrument is invalid, order is duplicated, strategy is disabled, kill switch is active or required model validation is missing.

• Risk checks must run server-side and cannot be bypassed from the frontend.

18. Order Engine

```
Order state machine:
CREATED → VALIDATING → SUBMITTED → OPEN
→ PARTIALLY_FILLED → FILLED → CLOSED

Alternative states:
REJECTED / CANCELLED / FAILED / EXPIRED
```

• Use client correlation IDs and idempotency keys. Store broker order IDs. Handle retries carefully so the same logical order is not duplicated.

19. Broker Adapter Layer

```
Standard interface:
authenticate()
get_profile()
get_funds()
get_positions()
get_holdings()
get_orders()
get_trades()
get_quote()
get_historical_data()
place_order()
modify_order()
cancel_order()
reconcile()
```

• Implement adapters for Dhan, Upstox and Groww. Keep provider-specific code isolated from strategy/risk/business logic.

20. Broker Reconciliation

• Regularly compare local orders with broker orders, local trades with broker trades, local positions with broker positions and local funds with broker funds.

• Create audit events for discrepancies. Never silently overwrite unexplained differences.

• After disconnects, reconnect and reconcile before allowing new live orders.

21. Portfolio, Orders & Reports

• Provide portfolio holdings, open/closed positions, orders, trades, P&L;, realized/unrealized P&L;, risk exposure and historical snapshots.

• Generate reports for daily/weekly/monthly performance and trade history.

• Add notifications for fills, rejections, risk blocks, broker disconnects, stale data, kill-switch activation and system failures.

22. Activity & Audit Logs

• Record authentication events, broker connection events, strategy changes, model changes, risk blocks, order lifecycle events, admin actions and system errors.

• Audit records should include actor, action, timestamp, correlation ID and relevant before/after metadata where appropriate.

23. Workers & Background Jobs

• Use background workers for market-data ingestion, indicator calculations, backtests, ML training, reconciliation, reports, notifications and cleanup jobs.

• Do not perform heavy training/backtesting synchronously in normal API requests.

24. API & WebSocket Design

• Create versioned REST APIs for auth, users, brokers, market data, instruments, watchlists, screeners, strategies, models, backtests, paper trading, live trading, orders, positions, portfolio, reports, notifications and admin.

• Create authenticated WebSocket channels for market updates, orders, positions, P&L;, system health and notifications.

• Document request/response schemas and errors.

25. Frontend State & UX

• Use a predictable state-management/query approach and avoid duplicating server state unnecessarily.

• Build responsive layouts with sidebar/topbar/navigation, searchable tables, filters, charts, drawers/modals and confirmation dialogs.

• Show skeleton loaders, empty states, retry states, error states and unavailable-data states.

• Do not show fake numbers merely to make the UI look populated.

26. Safety & Live Trading Lock

• Live trading is OFF by default.

• Require explicit activation only after broker credentials, API requirements, instrument mapping, market-data health, risk settings, validation and compliance checks are complete.

• Provide a server-side kill switch.

• Never promise profits or guaranteed returns.

• Never automatically place real-money trades during development or testing.

27. External API Rule

• Before implementing live broker functionality, verify the current official Dhan, Upstox and Groww API documentation, authentication requirements, order restrictions, WebSocket requirements, rate limits, pricing and applicable Indian regulatory/compliance requirements.

• Do not invent endpoints, credentials, API fields or capabilities. If a provider feature is unavailable, show a clear configuration/unavailable state.

28. Error & Failure Handling

• Do not fabricate broker responses, market prices, P&L;, model accuracy or trade fills.

• Handle API failures, timeouts, malformed responses, stale feeds, Redis failure, MySQL failure, worker failure and WebSocket disconnects safely.

• If MySQL is unavailable, enter safe mode. If Redis is required for a trading-critical operation and unavailable, stop dependent trading actions safely.

29. Testing

• Create unit, integration, API, database, strategy, backtest, risk, authentication, broker-adapter and WebSocket tests.

• Security tests should include SQL injection, XSS, CSRF, brute force, credential stuffing, session fixation, token reuse, OAuth state validation, open redirect, rate-limit bypass, IDOR and broken access control.

• Paper-trading tests must cover buy/sell, SL, target, partial fills, cancel, reject, slippage, fees, daily loss, max positions, max trades and kill switch.

• Data tests must cover missing/duplicate/stale data, wrong instruments, timezone issues, holidays and corporate actions.

30. Deployment

• Frontend must be Vercel-compatible. Backend and workers must be Docker-compatible.

• Use managed MySQL and Redis. Configure HTTPS, domain, environment variables and secret management.

• Provide separate development, staging and production configurations.

• Never commit .env files, API keys, client secrets or broker tokens.

31. Initial Strategy Example

• Implement a clearly labeled example strategy for development/paper testing only: EMA20 > EMA50, RSI > 50, Close above 20-day high and RVOL > 1.5; stop based on ATR and target using 1:3 RR.

• Do not represent this example as a profitable or validated investment strategy.

32. First-Run User Flow

• Register → verify email → enable optional 2FA → configure profile → select paper mode → configure starting capital → view market-data status → create/import strategy → run backtest → review validation → launch paper trading → monitor risk/P&L; → only later request live-trading activation.

33. Final Deliverables

• Deliver complete source code, frontend, backend, database migrations, seed/configuration tools where appropriate, workers, broker adapters, WebSocket implementation, strategy builder, screener, backtesting engine, ML lab/model registry, paper trading, risk engine, order engine, audit logs, tests, Docker configuration, environment templates and documentation.

• Include README with setup commands, architecture, environment variables, database setup, Redis setup, API documentation, testing commands, deployment instructions and troubleshooting.

• Include a clear list of features that require external credentials or paid/current broker services.

34. Final Acceptance Checklist

• Confirm that all pages are routed and functional.

• Confirm that data is backend-driven.

• Confirm that no fake P&L;/orders/market prices are used.

• Confirm that risk checks are server-side.

• Confirm that paper trading works end-to-end.

• Confirm that backtests run asynchronously.

• Confirm that ML training is separated from execution.

• Confirm that broker adapters are isolated and current official docs are followed.

• Confirm that order idempotency and reconciliation exist.

• Confirm that audit logs exist.

- Confirm that security and authorization tests exist.
- Confirm that live trading remains locked unless explicitly configured and approved.
- Fix build, type, migration and test errors before declaring the project complete.